

18/06/2026

SQL Date Functions and Date Math

Extracting Date Parts:

01. Day Name from Order Date

orders

order_id	customer_id	order_date
1001	101	2026-05-01
1002	102	2026-05-02
1003	103	2026-05-03
1004	104	2026-05-04
1005	105	2026-05-05

01. Return each order with the day name of the order date.

USE: DAYNAME AS

Expected Columns: order_id, customer_id, order_date, day-name

```
SELECT order_id,  
       customer_id,  
       order_date,  
       DAYNAME (order_date) AS day-name  
FROM orders;
```

Output

order_id	customer_id	order_date	day-name
1001	101	2026-05-01	Friday
1002	102	2026-05-02	Saturday
1003	103	2026-05-03	Sunday
1004	104	2026-05-04	Monday
1005	105	2026-05-05	Tuesday

Month Name from Signup Date

customer-signups		
customer_id	customer_name	signup_date
201	John	2026-01-15
202	Mary	2026-02-20
203	Peter	2026-03-05
204	Sarah	2026-04-18
205	Thabo	2026-05-30

02. Return each customer with the name of the month in which they signed up,

Use: MONTHNAME & AS

Expected columns: customer_id, customer_name, signup_date, signup-month_name

```
SELECT customer_id,
       customer_name,
       signup_date,
       MONTHNAME(signup_date) AS signup-month_name
FROM customer-signups;
```

output			
Customer_id	customer_name	signup_date	signup-month_name
201	John	2026-01-15	January
202	Mary	2026-02-20	February
203	Peter	2026-03-05	March
204	Sarah	2026-04-18	April
205	Thabo	2026-05-30	May

Month Number from Sale Date

Sales

sale_id	product_name	sale_date	amount
S001	Laptop	2026-01-10	12000
S002	Mouse	2026-02-15	250
S003	Keyboard	2026-03-20	700
S004	Monitor	2026-04-25	3500
S005	Desk	2026-05-30	2000

Q3. Return each sale with the month number extracted from sales
use: MONTH & AS

Expected Columns: sale_id, product_name, sale_date, sale_month

```
SELECT sale_id,  
       product_name,  
       sale_date,
```

```
       MONTH(sale_date) AS sale_month
```

```
FROM sales;
```

output

sale_id	product_name	sale_date	sale_month
S001	Laptop	2026-01-10	1
S002	Mouse	2026-02-15	2
S003	Keyboard	2026-03-20	3
S004	Monitor	2026-04-25	4
S005	Desk	2026-05-30	5

Year from Transaction Date

transactions			
transaction_id	customer_id	transaction_date	amount
T001	101	2024-12-15	500
T002	102	2025-01-20	1200
T003	103	2025-06-10	800
T004	104	2026-02-05	1500
T005	105	2026-05-25	2000

Q4. Return each transaction with the year extracted from transaction_date
 use: YEAR & AS

Expected Columns: transaction_id, customer_id, transaction_date, transaction_year

```
SELECT transaction_id,
       customer_id,
       transaction_date,
       YEAR(transaction_date) AS transaction_year
FROM transactions;
```

output			
transaction_id	customer_id	transaction_date	transaction_year
T001	101	2024-12-15	2024
T002	102	2025-01-20	2025
T003	103	2025-06-10	2025
T004	104	2026-02-05	2026
T005	105	2026-05-25	2026

Day of Month from Delivery Date

deliveries		
delivery_id	customer_id	delivery_date
D001	101	2026-05-01
D002	102	2026-05-08
D003	103	2026-05-15
D004	104	2026-05-22
D005	105	2026-05-29

Q5. Return each delivery with the day of the month extracted from delivery_date
use: DAY & AS

Expected Columns: delivery_id, customer_id, delivery_date, day-of-month

```
SELECT delivery_id,  
       customer_id,  
       delivery_date,  
       DAY (delivery_date) AS day_of_month  
FROM deliveries;
```

output			
delivery_id	customer_id	delivery_date	day_of_month
D001	101	2026-05-01	1
D002	102	2026-05-08	8
D003	103	2026-05-15	15
D004	104	2026-05-22	22
D005	105	2026-05-29	29

Add Today's Date,

employees		
employee-id	employee-name	department
301	Nandi	Sales
302	Brian	IT
303	Lerato	Finance
304	Sipho	HR
305	Aisha	Marketing

06. Return all employees and add a column showing today's date.

USE: CURRENT() AS

Expected columns: employee-id, employee-name, department, today-date

```
SELECT employee-id,
       employee-name,
       department,
       CURRENT-DATE() AS today-date
FROM employees;
```

output			
employee-id	employee-name	department	today-date
301	Nandi	Sales	2026-06-18
302	Brian	IT	2026-06-18
303	Lerato	Finance	2026-06-18
304	Sipho	HR	2026-06-18
305	Aisha	Marketing	2026-06-18

Convert Text to Date

online_orders		
order_id	customer_id	order_date_text
4001	101	2026-01-15
4002	102	2026-02-20
4003	103	2026-03-25
4004	104	2026-04-10
4005	105	2026-05-05

07. Convert the order_date_text string column into a proper date column.
USE 'TO_DATE & AS

Expected Columns: order_id, customer_id, order_date_text, order_date.

```
SELECT order_id,
       customer_id,
       order_date_text,
       TO_DATE (order_date_text) AS order_date
FROM online_orders;
```

Output			
order_id	customer_id	order_date_text	order_date
4001	101	2026-01-15	2026-01-15
4002	102	2026-02-20	2026-02-20
4003	103	2026-03-25	2026-03-25
4004	104	2026-04-10	2026-04-10
4005	105	2026-05-05	2026-05-05

Format Date as Text

payment_dates		
payment_id	customer_id	payment_date
P001	101	2026-01-05
P002	102	2026-02-10
P003	103	2026-03-15
P004	104	2026-04-20
P005	105	2026-05-25

Q8. Format payment_date as a text string in the format YYYY-MM-DD

USE : TO_CHAR & AS

Expected Columns: payment_id, customer_id, payment_date, formatted_payment_date.

```
SELECT payment_id,
       customer_id,
       payment_date,
       TO_CHAR(payment_date, 'YYYY-MM-DD') AS formatted_payment_date
FROM payment_dates;
```

Output			
payment_id	customer_id	payment_date	formatted_payment_date
P001	101	2026-01-05	2026-01-05
P002	102	2026-02-10	2026-02-10
P003	103	2026-03-15	2026-03-15
P004	104	2026-04-20	2026-04-20
P005	105	2026-05-25	2026-05-25

Days Since Last Purchase

customer_purchases		
customer_id	customer_name	Last_purchase_date
501	John	2026-05-01
502	Mary	2026-05-10
503	Peter	2026-05-15
504	Sarah	2026-05-20
505	Thabo	2026-05-25

09. Calculate how many days have passed since each customer's last purchase. Use today's date.

USE: CURRENT_DATE(), DATEIF & AS

```
SELECT customer_id,
       customer_name,
       last_purchase_date
```

```
DATEIF(CURRENT_DATE(), last_purchase_date) AS days_since_last_purchase
FROM customer_purchases;
```

output using 20/06/28

customer_id	customer_name	last_purchase_date	days_since_last_purchase
501	John	2026-05-01	36
502	Mary	2026-05-10	31
503	Peter	2026-05-15	26
504	Sarah	2026-05-20	19
505	Thabo	2026-05-25	10

Expected Delivery Date

Shipping-orders

order_id	customer_id	order_date
6001	101	2026-05-01
6002	102	2026-05-03
6003	103	2026-05-05
6004	104	2026-05-07
6005	105	2026-05-09

10. Calculate the expected delivery date by adding 7 days to the order_date.

USE: DATEADD & AS

```
SELECT order_id,
       customer_id,
       order_date,
```

```
       DATEADD(day, 7, order_date) AS expected_delivery_date
FROM shipping-orders;
```

Output

order_id	customer_id	order_date	expected_delivery_date
6001	101	2026-05-01	2026-05-08
6002	102	2026-05-03	2026-05-10
6003	103	2026-05-05	2026-05-12
6004	104	2026-05-07	2026-05-14
6005	105	2026-05-09	2026-05-16

Year, Month, Day Separately

bookings		
booking_id	customer_id	booking_date
B001	101	2026-01-12
B002	102	2026-02-18
B003	103	2026-03-22
B004	104	2026-04-09
B005	105	2026-05-27

1. Extract the year, month number, and day from booking_date into the separate columns.

USE: YEAR, MONTH, DAY & AS.

```
SELECT booking_id,  
       customer_id,  
       booking_date,  
       YEAR(booking_date) AS booking_year,  
       MONTH(booking_date) AS booking_month,  
       DAY(booking_date) AS booking_day  
FROM bookings;
```

output					
booking_id	customer_id	booking_date	booking_year	booking_month	booking_day
B001	101	2026-01-12	2026	1	12
B002	102	2026-02-18	2026	2	18
B003	103	2026-03-22	2026	3	22
B004	104	2026-04-09	2026	4	09
B005	105	2026-05-27	2026	5	27

Filter Orders from 2026

yearly_orders			
order_id	customer_id	order_date	amount
7001	101	2024-12-15	500
7002	102	2025-01-20	1200
7003	103	2025-06-10	800
7004	104	2026-02-05	1500
7005	105	2026-05-25	2000

12. Return only orders from the year 2026.
USE: YEAR, WHERE & AS

```
SELECT order_id,  
       customer_id,  
       order_date,  
       YEAR(order_date) AS order_year,  
       amount
```

```
FROM yearly_orders  
WHERE YEAR(order_date) = 2026;
```

Output

order_id	customer_id	order_date	order_year	amount
7004	104	2026-02-05	2026	1500
7005	105	2026-05-25	2026	2000

Filter Orders from March

monthly-orders			
order_id	customer_id	order_date	amount
8001	101	2026-01-15	500
8002	102	2026-02-20	1200
8003	103	2026-03-10	800
8004	104	2026-03-25	1500
8005	105	2026-05-30	2000

13. Return only orders placed in March (month number 3).
USE: MONTH, WHERE $\neq$ AS

```
SELECT order_id,
       customer_id,
       order_date,
       MONTH (order_date) AS order_month,
       amount
```

```
FROM monthly-orders
WHERE MONTH (order_date) = 3;
```

Output

order_id	customer_id	order_date	order_month	amount
8003	103	2026-03-10	3	800
8004	104	2026-03-25	3	1500

Last Day of the Month

Subscriptions		
subscription_id	customer_id	start_date
SUB001	101	2026-01-10
SUB002	102	2026-02-15
SUB003	103	2026-03-20
SUB004	104	2026-04-25
SUB005	105	2026-05-30

14. Return the last day of the month for each subscription start date.

USE : LAST_DAY & AS

```
SELECT subscription_id,
       customer_id,
       start_date,
       LAST_DAY(start_date) AS month_end_date
FROM subscriptions;
```

output			
subscription_id	customer_id	start_date	month_end_date
SUB001	101	2026-01-10	2026-01-31
SUB002	102	2026-02-15	2026-02-28
SUB003	103	2026-03-20	2026-03-31
SUB004	104	2026-04-25	2026-04-30
SUB005	105	2026-05-30	2026-05-31

First Day of the Month

Campaign_sends		
send_id	customer_id	send_date
C001	101	2026-01-12
C002	102	2026-02-18
C003	103	2026-03-22
C004	104	2026-04-09
C005	105	2026-05-27

15. Return the first day of the month for each campaign send date.

USE : DATE_TRUNC & AS

```

SELECT send_id,
       customer_id,
       send_date,
       DATE_TRUNC('month', send_date) AS month_start_date
FROM campaign_sends;
    
```

output

Send_id	customer_id	send_date	month_start_date
C001	101	2026-01-12	2026-01-01
C002	102	2026-02-18	2026-02-01
C003	103	2026-03-22	2026-03-01
C004	104	2026-04-09	2026-04-01
C005	105	2026-05-27	2026-05-01

FORMAT as Month and Year

invoice_dates		
invoice_id	customer_id	invoice_date
INV001	101	2026-01-05
INV002	102	2026-02-10
INV003	103	2026-03-15
INV004	104	2026-04-20
INV005	105	2026-05-25

16. Format invoice_date as a text string showing month name and year. Example January 2026.
USE 'TO_CHAR' & 'AS'

```
SELECT invoice_id,
       customer_id,
       invoice_date,
       TO_CHAR (invoice_date, 'Month YYYY') AS invoice_month_year
FROM invoice_dates;
```

output			
invoice_id	customer_id	invoice_date	invoice_month_year
INV001	101	2026-01-05	January 2026
INV002	102	2026-02-10	February 2026
INV003	103	2026-03-15	March 2026
INV004	104	2026-04-20	April 2026
INV005	105	2026-05-25	May 2026

Customer Age from Date of Birth

customer_birthdays		
customer_id	customer_name	date_of_birth
901	John	1998-05-10
902	Mary	1990-08-20
903	Peter	2002-03-15
904	Sarah	1985-12-01
905	Thabo	2000-07-30

7. Calculate each customer's age in years using their date of birth and today's date.
 USE: CURRENT_DATE(), DATEIFF or YEAR & AS

```
SELECT customer_id,
       customer_name,
       date_of_birth,
       YEAR(2026-06-20CURRENT_DATE()) - YEAR(date_of_birth) AS customer_age
FROM customer_birthdays;
```

Output			
customer_id	customer_name	date_of_birth	customer_age
901	John	1998-05-10	28
902	Mary	1990-08-20	36
903	Peter	2002-03-15	24
904	Sarah	1985-12-01	41
905	Thabo	2000-07-30	26

Classify Weekend vs Weekday

weekend_orders

order_id	customer_id	order_date
9001	101	2026-05-01
9002	102	2026-05-02
9003	103	2026-05-03
9004	104	2026-05-04
9005	105	2026-05-05

18. Classify each order as Weekend or Weekday based on the day name of the order date. Saturday and Sunday = Weekend
All others = Weekday.

USE: DAYNAME, CASE, AS

```

SELECT order_id,
       customer_id,
       order_date,
       DAYNAME(order_date) AS day-name,
       CASE
         WHEN DAYNAME(order_date) IN ('Saturday', 'Sunday')
           THEN 'weekend'
         ELSE 'weekday'
       END AS day-type
FROM weekend_orders;
    
```

output

order_id	customer_id	order_date	day-name	day-type
9001	101	2026-05-01	Friday	Weekday
9002	102	2026-05-02	Saturday	Weekend
9003	103	2026-05-03	Sunday	Weekend
9004	104	2026-05-04	Monday	Weekday
9005	105	2026-05-05	Tuesday	Weekday

Transactions by Quarter

Quarterly_transactions

transaction_id	customer_id	transaction_date	amount
Q001	101	2026-01-15	500
Q002	102	2026-03-20	1200
Q003	103	2026-04-10	800
Q004	104	2026-07-05	1500
Q005	105	2026-10-25	2000

19. Extract the quarter number from each transaction date.
USE: QUARTER & AS

```
# SELECT transaction_id,
      customer_id,
      transaction_date,
      QUARTER(transaction_date) AS transaction_quarter,
      amount
FROM quarterly_transactions;
```

Output

transaction_id	customer_id	transaction_date	transaction_quarter	amount
Q001	101	2026-01-15	1	500
Q002	102	2026-03-20	1	1200
Q003	103	2026-04-10	2	800
Q004	104	2026-07-05	3	1500
Q005	105	2026-10-25	4	2000

Orders Older Than 30 Days

recent_orders			
order_id	customer_id	order_date	amount
R001	101	2026-04-01	500
R002	102	2026-04-15	1200
R003	103	2026-05-01	800
R004	104	2026-05-10	1500
R005	105	2026-05-25	2000

20. Return only orders that are more than 30 days old from today. Calculate days_since_order using today's date, USE: CURRENT_DATE(), DATEIFF, WHERE, AS

```
SELECT order_id,
       customer_id,
       order_date,
       amount,
```

```
       DATEIFF('2026-06-20', order_date) AS days_since_order
FROM recent_orders
WHERE DATEIFF('2026-06-20', order_date) > 30;
```

output				
order_id	customer_id	order_date	amount	days_since_order
R001	101	2026-04-01	500	80
R002	102	2026-04-15	1200	76
R003	103	2026-05-01	800	50
R004	104	2026-05-10	1500	41
R005	105	2026-05-25	2000	26

Customer Recency Segment

customer_recency			
customer_id	customer_name	last_purchase_date	total_spend
1001	John	2026-05-25	5000
1002	Mary	2026-05-10	2500
1003	Peter	2026-04-01	700
1004	Sarah	2026-02-15	15000
1005	Thabo	2025-12-20	300

B. Calculate days_since_last_purchase for each customer using CURRENT_DATE. Then use CASE to classify ≤ 30 days = Active Customer. 31-90 days = At Risk Customer, > 90 days = Inactive Customers.

USE: CURRENT_DATE(), DATEIFF, CASE, AS

```

SELECT customer_id,
       customer_name,
       last_purchase_date,
       DATEIFF('2026-06-20', last_purchase_date) AS days_since_last_purchase
CASE
    WHEN DATEIFF('2026-06-20', last_purchase_date)  $\leq 30$ 
    THEN 'Active Customer'
    WHEN DATEIFF('2026-06-20', last_purchase_date) BETWEEN 31 AND 90
    THEN 'At Risk Customer'
    ELSE 'Inactive Customer'
END AS customer_status
FROM customer_recency;
    
```


output

customer_id	customer_name	last_purchase_date	days_since_last_purchase	customer_status
1001	John	2026-05-25	26	Active customer
1002	Mary	2026-05-10	41	At Risk
1003	Peter	2026-04-01	80	At Risk
1004	Sarah	2026-02-15	125	Inactive
1005	Thabo	2025-12-20	182	Inactive